

План захисту · Розробник 2

Підсистема каталогу контенту, агрегатора метаданих та потокового відео

Дослідження / Аналіз

Перед розробкою підсистеми було проаналізовано існуючі рішення та обмеження ринку. Більшість популярних metadata-API, таких як Marvel і ComicVine, вимагають ключі доступу або підпис кожного запиту, що ускладнює їх використання.

Тому як основа були обрані безкоштовні no-auth джерела (Jikan, Superhero API, TVMaze), які дозволяють працювати без реєстрації.

Додатково було враховано підхід Cache-Aside для зменшення кількості зовнішніх запитів і обходу rate-limit'ів, а також UX-патерни Netflix (рейли, hero-карусель, глобальний пошук через Cmd/Ctrl+K). Для відео використовувався стандарт HTML5, що дозволяє відтворення без сторонніх плеєрів і SDK.

Актуальність і проблема підсистеми

Основною проблемою було те, що платформа не мала власної бази контенту і змушена була покладатися на зовнішні джерела даних. Більшість із них або платні, або мають жорсткі обмеження — потрібна авторизація, підпис запитів і ліміти на кількість звернень.

І одразу, без паузи — додатково ускладнювало ситуацію те, що різні API повертають дані в різних форматах, через що комікси і манга не могли одразу виглядати однаково в одному каталозі.

Мета і задачі підрозділу

Мета підсистеми — зробити стабільний і відмовостійкий контентний шар платформи, який працює на основі безкоштовних відкритих джерел і кешування.

Для цього було реалізовано агрегатор метаданих, інтегровано кілька API (Jikan, Superhero API, TVMaze), а також додано кешування для зменшення навантаження і прискорення роботи.

Окремо було вирішено задачі фільтрації контенту, уніфікації відображення карток і побудови єдиного каталогу. Поверх цього реалізовано пошук у стилі Netflix, hero-карусель і HTML5-плеєр для перегляду відео.

1. Агрегатор метаданих

Було реалізовано єдиний `MetadataController`, який об'єднує різні джерела даних через сервіси. Завдяки цьому клієнт працює тільки з уніфікованим DTO і не залежить від конкретних провайдерів. Це дозволяє в майбутньому змінити або замінити API без змін на фронтенді.

→ Тут показую `Controllers/MetadataController.cs`, рядки 24, 45, 99, 122 — це атрибути маршрутів `top-anime`, `search`, `superheroes`, `superhero-shows`. Кажу: «Усього у файлі 16 ендпоінтів. Клієнт працює тільки з ними і не знає, чи стоїть за ними Jikan, akabab чи TVMaze».

→ Потім швидко переключаюся на `Services/JikanService.cs`, на сигнатуру класу: `public class JikanService : IJikanService`. Кажу: «Контролер працює з інтерфейсом, а не з конкретним сервісом. У майбутньому, коли підв'яжемо ліцензовану БД, досить буде замінити реалізацію інтерфейсу — контролер і фронтенд лишаються без змін».

2. Кешування та оптимізація

Впроваджено `Cache-Aside` з різними TTL і загальний механізм пагінації. Це дозволило значно зменшити кількість зовнішніх запитів і зняти залежність від `rate-limit`'ів. У результаті повторні запити обробляються практично миттєво.

→ Залишаюсь у `Services/JikanService.cs`. Прокручую до рядків 29–53 — сигнатура `generic`-методу `GetPagedCached`. Кажу: «Один типобезпечний `generic`-метод покриває близько десяти різних DTO Jikan-API. Без нього було б десять майже однакових копій — а тут усі зміни логіки кешування в одному місці».

→ У тому самому файлі прокручую вгору до рядка 10 — константа `ListCacheTtl = TimeSpan.FromMinutes(15)`. Кажу: «Ось конкретний TTL для списків — 15 хвилин. Завдяки цьому повторні переходи каталогом не йдуть у мережу, `rate-limit Jikan` — три запити в секунду — не вичерпується навіть під час інтенсивної демонстрації».

→ І в тому самому файлі — рядок 77, константа `MangaExcludedGenres = "9,12,28,30,49"`. Кажу: «Поки ми тут — маленький, але важливий момент: додаткова фільтрація NSFW-жанрів манги, бо параметр `sfw=true` самого `Jikan` виявився недостатнім».

3. Bulk-дані (akabab)

Один великий датасет завантажується одним запитом і кешується, після чого пошук і фільтрація працюють повністю в пам'яті. Це прибрало мережеві запити під час взаємодії з каталогом і зробило роботу сторінки незалежною від доступності API.

→ Переключаюся на `Services/SuperheroService.cs`, рядок 118 — сигнатура методу `LoadAllAsync` з одним викликом `HttpClient.GetAsync`. Кажу: «Один `fetch` — близько одного мегабайта JSON, приблизно 731 запис. Викликається раз на 12 годин, далі все в пам'яті».

→ У тому самому файлі прокручую вгору, рядок 20 — константа `CharactersCacheTtl = TimeSpan.FromHours(12)`. Кажу: «А це сам TTL — 12 годин, тобто за добу — максимум два запити до `akabab`».

→ У тому самому файлі трохи нижче — рядки 58–70, шматок з LINQ-фільтром `characters.Where(h => h.Biography.Publisher == publisher)`. Кажу: «А це сам пошук і фільтрація — звичайний LINQ по списку в пам'яті. Тому перемикання `Marvel` → `DC` → `Image` миттєве, без жодного мережевого запиту».

4. Уніфікація UI

Було створено універсальний компонент карток і єдину систему відображення контенту. Різні типи даних виглядають однаково, а нові типи можна додавати

без дублювання коду. Навіть відсутні обкладинки не ламають інтерфейс завдяки fallback.

→ Переключаюся на `components/MediaCard.tsx`, рядки 1–25 — тільки тип `MediaCardProps` з полями `title`, `image`, `kind`, `onClick`. Кажу: «Один компонент з чотирма ключовими props — він рендерить однаково що комікс, що мангу. Завдяки полю `kind` правильно реагуємо на клік».

→ У тому самому файлі прокручую в тіло компонента, рядок 46 — JSX з `setBroken(true)}` і fallback-блок з градієнтом. Кажу: «Деталь, що рятує UI: при будь-якій помилці завантаження обкладинки рендериться градієнт із назвою. Сітка карток ніколи не ламається».

→ Швидко переключаюся на `components/Rail.tsx`, рядок 79 — `className` з `no-scrollbar`. Кажу: «Netflix-стиль рейла — горизонтальний скрол з прихованим скролбаром. Один utility-клас без зайвих залежностей».

5. Пошук і відео

Реалізовано глобальний пошук по трьох джерелах, hero-карусель і HTML5-плеєр. Це дозволило повністю закрити сценарій «знайти → відкрити → переглянути» всередині SPA без сторонніх SDK.

→ Переключаюся на `components/MasterSearch.tsx`, рядки 35–37 — три `useQuery` поспіль: `comicsQuery`, `heroesQuery`, `mangaQuery`. Кажу: «Дебаунс 250 мс, потім три паралельні запити одночасно. Клієнт бачить результати трьох категорій в одному кадрі».

→ У тому самому файлі прокручую нижче, рядки 52–54 — `onClick` результату викликає `setOpenModal(...)` без `navigate(...)`. Кажу: «Клік по результату відкриває модальку прямо в накладенні, без переходу на іншу сторінку — користувач не втрачає контекст».

→ Переключаюся на `components/TVHero.tsx`, рядки 12–13 (константа `CLIP_DURATION` на 10–15 секунд) і нижче 32–38 (`useEffect` з `Math.random()`). Кажу: «Hero-карусель програє трейлери випадковими фрагментами по 10–15 секунд — кожне відвідування показує інший шматок іншого трейлера».

→ Останнє — `pages/Watch.tsx`, тільки JSX-тег . Кажу: «Нативний HTML5 video. Жодного зовнішнього SDK — розмір клієнтського бандла не зростає за рахунок плеєра».

Підсумок

Реалізовано повний контент-шар платформи: агрегатор з трьох безкоштовних API в одній точці входу, кешування з диференційованими TTL, bulk-стратегія для великого датасету з in-memory фільтрацією, уніфікований UI з fallback, in-place пошук по трьох джерелах і потокове відео. Сценарій «знайти → відкрити → переглянути» закрито повністю в межах SPA, без сторонніх SDK.

Резерв для питань

Якщо запитують, що буде, якщо Jikan або akabab лежить — відкриваю `Services/SuperheroService.cs`, `try/catch` навколо HTTP-запиту в `LoadAllAsync`. Кажу: «Graceful-fallback: помилка → лог і порожній список. Сервіс не падає, сторінка лишається живою».

— Чому саме ці три API: безкоштовні, без авторизації, з обкладинками. Платні і ключові (Marvel, ComicVine, TMDb) відкинуті свідомо.

— Навіщо `GetPagedCached`: прибирає дублювання між ~10 майже однаковими методами, типобезпечно.

— Як працює пошук по трьох джерелах: дебаунс 250 мс, три паралельні `useQuery`, результати згруповано, клік відкриває модальку in-place.

— Чому akabab тягнеться весь одразу: статичний файл ~1 МБ; один `fetch` + 12 годин кешу дешевше, ніж посторінкові запити, і знімає залежність від доступності джерела.

— Як легко буде підставити справжню ліцензовану БД: замінюються тільки реалізації `IJikanService` і `ISuperheroService` — контролери і клієнт лишаються незмінні.